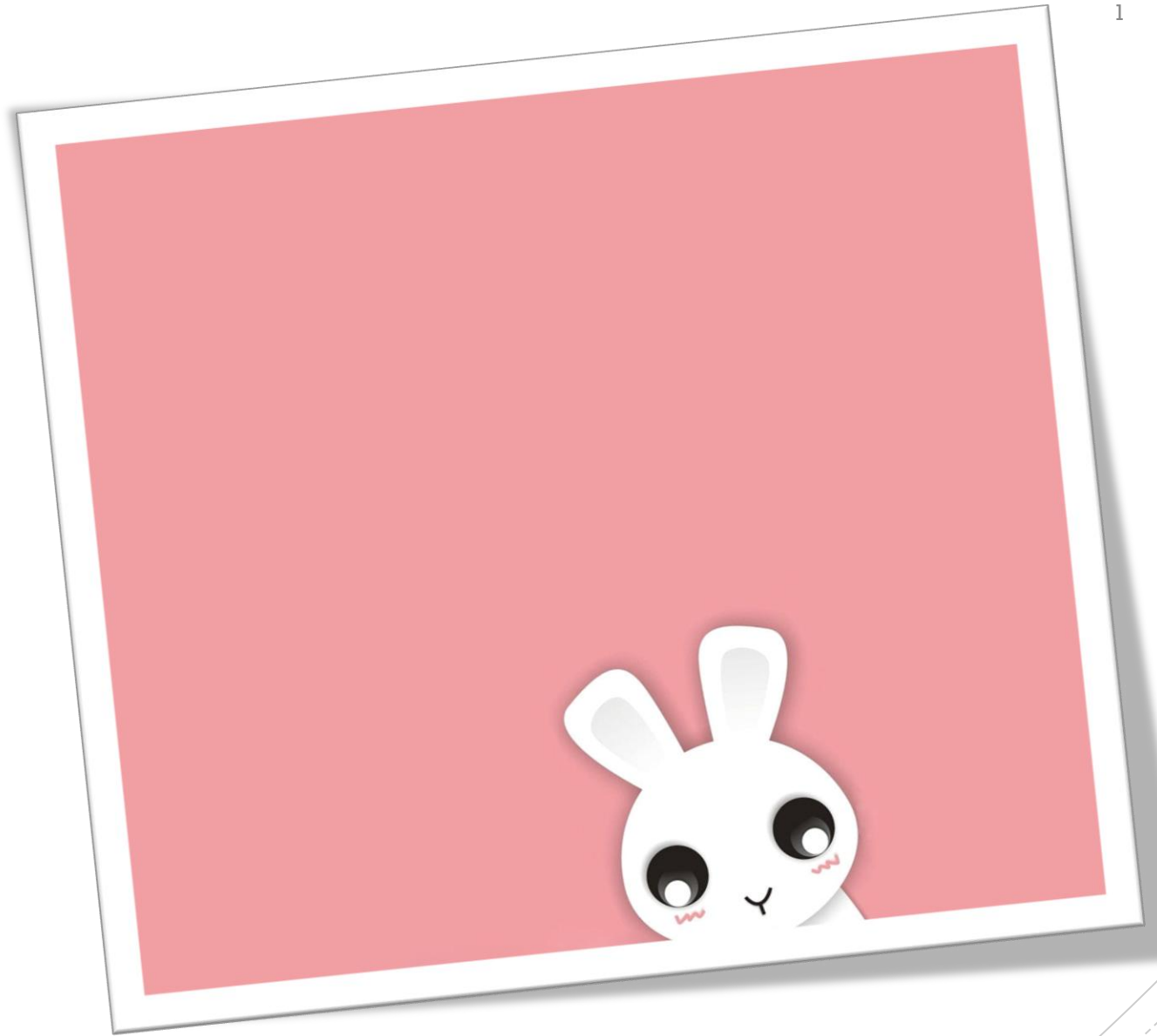
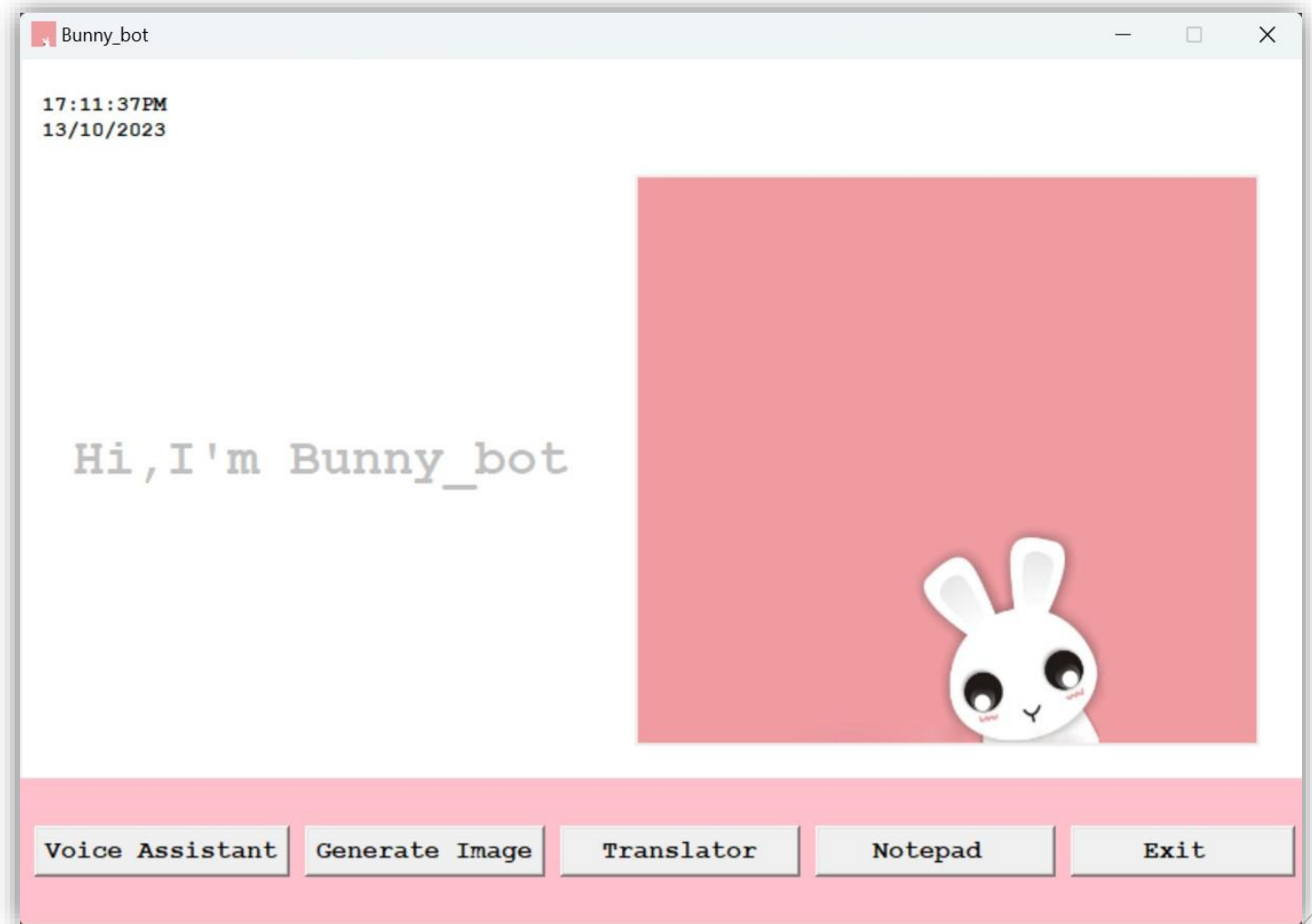


Bunny_bot:
Your Multifunctional
AI Assistant



Project topic:
Artificial intelligence assistant
Project title:
Bunny_bot
Supervisor:
Marzieh Dadvar
Producer:
Hannah Fathi



Introduction

Not so long ago, assistants were used in all fields to help get things done and organize these things. These assistants performed many tasks, including setting work appointments, remembering these appointments, searching for information, etc., and today, with the spread of artificial intelligence assistant technology, the use of these types of assistants has grown significantly compared to physical assistants. These assistants are able to answer many of the tasks of physical assistants. In this project, I am going to tell you how artificial intelligence assistant can help improve your daily life.

Problem statement

Our modern lives are fast-paced and full of complexity. We juggle numerous daily tasks and often find ourselves with limited time, leading to decreased productivity and increased stress. We need tools to improve efficiency and reduce the stress associated with our daily routines. Bunny_bot is a multifunctional AI assistant that offers a range of tools to solve these everyday challenges. It empowers us with features such as speech-to-text, language translation, text-to-image generation, note-taking, and much more.

Program features

Bunny_bot comes with an array of features, including:

- Voice Assistant
- Language Translation
- Text-to-Image Generation
- Note-Taking
- And more...

Architecture and technologies

Python libraries and technologies used (tkinter, speech recognition, pyttsx3, webbrowser, openai, threading, strptime.)

And used OpenAI API key in Python program to use OpenAI services.

Win1

Creating a tkinter window named win1 and setting up various elements within it. Here's a breakdown of what you're doing in this code:

1-Create a main tkinter window named win1 with a title "Bunny_bot," and set it to be non-resizable with a specific size. You also set an icon for the window.

2-Create a frame within the win1 window with some padding and a white background.

3-Load an image and display it in a label as the background.

4-Create a label displaying the text "Hi, I'm Bunny_bot" with a specific font and color.

5-Create a label for displaying the time (assuming there's a digital_time function to populate it).

6-Create a button frame with a pink background.

7-Create a list of buttons, each with a label and a corresponding function/command to be executed when the button is clicked.

8-Use a loop to create and display these buttons within the button_frame.

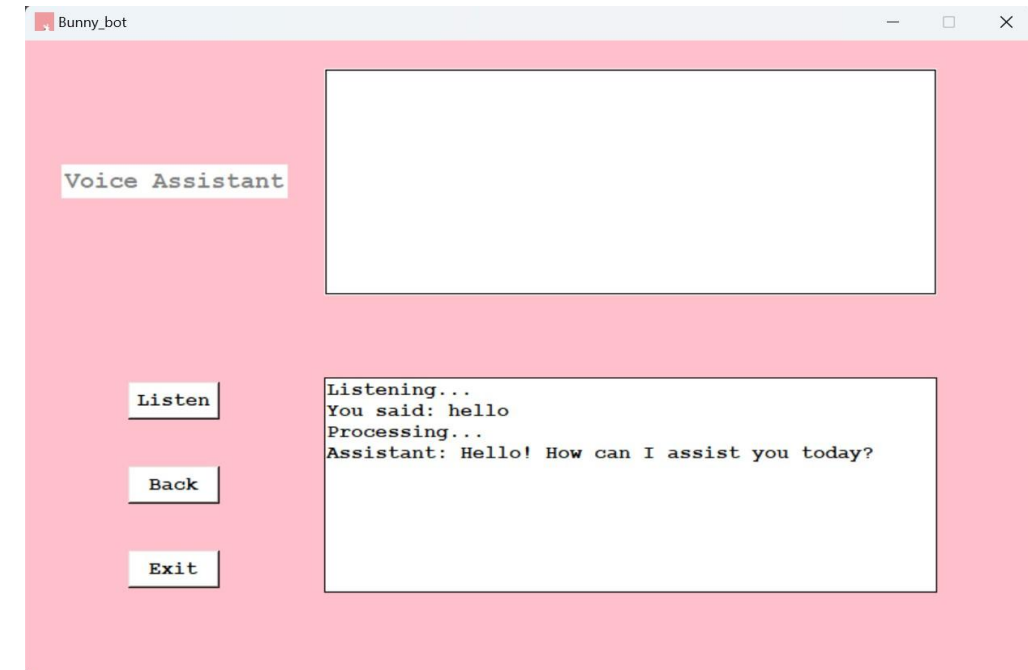
Overall, it appears you're creating a basic graphical user interface for your "Bunny_bot" application with buttons to switch to different functionalities or windows. Depending on your application, you would define the functions like `switch_to_window12`, `switch_to_window13`, etc., to handle the logic for each button's action.



Win2

Creating a second window (win2) for Bunny_bot application, specifically for the "Voice Assistant" functionality. Here's a breakdown of what's happening:

- 1-Create a new tkinter window (win2) with a title, fixed size, and a pink background. However, this window is initially hidden using win2.withdraw().
- 2-Create a Listbox widget named message_listbox for displaying the conversation messages. It's a white box with a specified font and dimensions.
- 3-Create a Text widget named response_text for displaying responses from the Voice Assistant. This text widget is also styled similarly to the message_listbox.
- 4-Create a label (win2_label) with the title "Voice Assistant," and a specific font and color.
- 5-Define a listen function that captures audio input from the microphone, processes it, and generates responses using OpenAI's GPT-3.5 model. It displays the conversation in message_listbox and the responses in response_text.



Win2

6-Define a `chat_with_openai` function that interacts with the OpenAI GPT-3.5 model by sending and receiving messages in a chat-like format.

7-Define a `speak` function to make the Voice Assistant read out the responses.

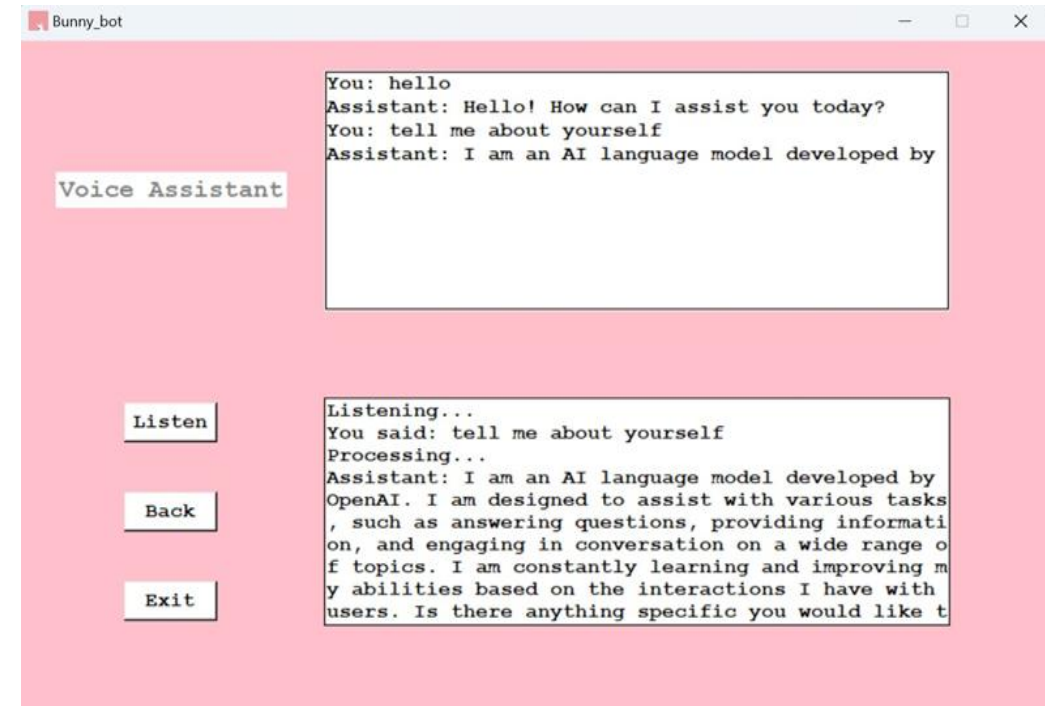
8-Create a frame (`frame2`) for organizing the buttons related to the "Voice Assistant" functionality.

9-Create a "Listen" button (`listen_button2`) that triggers the listen function.

10-Create a "Back" button (`back_button2`) to switch back to another window, presumably `switch_to_window21`.

11-Create an "Exit" button (`exit_button2`) to close `win2`.

12-Initialize an empty `chat_log` to keep track of the conversation messages.



Win3

Creating a third window (win3) for Bunny_bot application. This window is designed for two functionalities: opening web URLs and generating images from text prompts. Here's a breakdown of what's happening:

1-Create a new tkinter window (win3) with a title, fixed size, and a pink background. Similar to the previous window, this one is also initially hidden with win3.withdraw().

2-Define a function open_url() that opens a web URL in a browser when the user enters the URL and clicks the "Open" button.

3-Create a frame (entry_frame3) for organizing the entry widgets related to the URL input.

4-Create labels for the "Web Browser" title and "Enter URL" instructions.

5-Create an entry widget (url_entry) for the user to input the URL.

6-Create an "Open" button (open_button) to trigger the open_url() function.



Win3

7-Define a function `generate_image()` that generates an image based on user input (a text prompt) using OpenAI's image generation model.

8-Create a frame (`prompt_frame3`) for organizing the entry widgets related to text prompts.

9-Create labels for the "Generate Image" title and "Enter your prompt" instructions.

10-Create an entry widget (`prompt_entry`) for the user to input the text prompt.

11-Create a label (`result_label`) to display the generated image's URL.

12-Create a "Generate Image" button (`generate_button`) to trigger the `generate_image()` function.

13-Create a "View Image" button (`view_image_button`) to view the generated image. Initially, it's disabled until an image is generated.

14-Create "Back" and "Exit" buttons for navigating back to another window or exiting the application.



Win4

Creating a fourth window (win4) for Bunny_bot application, designed for text translation functionality. Here's a breakdown of what's happening:

1-Create a new tkinter window (win4) with a title, fixed size, and a pink background. This window is initially hidden with win4.withdraw().

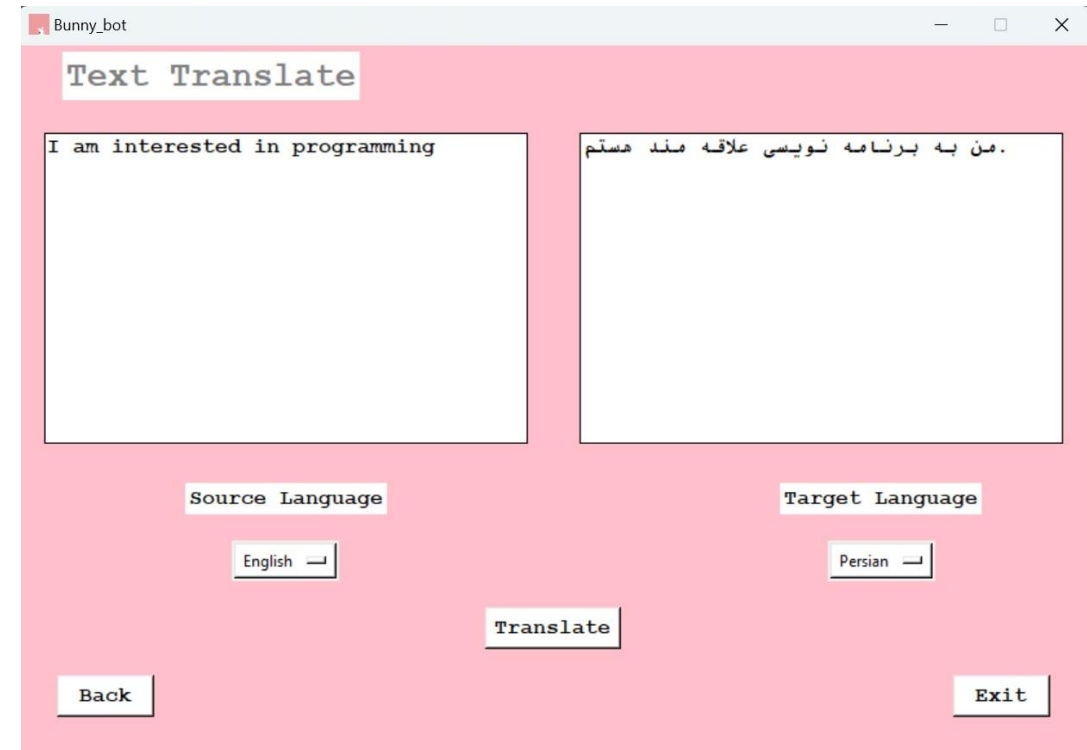
2-Define a function translate_text() that translates text from a source language to a target language. It uses OpenAI's API for this purpose.

3-Create a label (win4_label) with the title "Text Translate."

4-Create a Text widget (source_text_entry) for the user to input the source text. It's white with a specified font and size.

5-Create another Text widget (output_text) to display the translated text. It's also white with the specified font and size.

6-Create a label for selecting the source language.



Win4

7-Create a variable (source_language_var) to hold the selected source language.

8-Create an OptionMenu widget (source_language_menu) that allows the user to select the source language from a list of options.

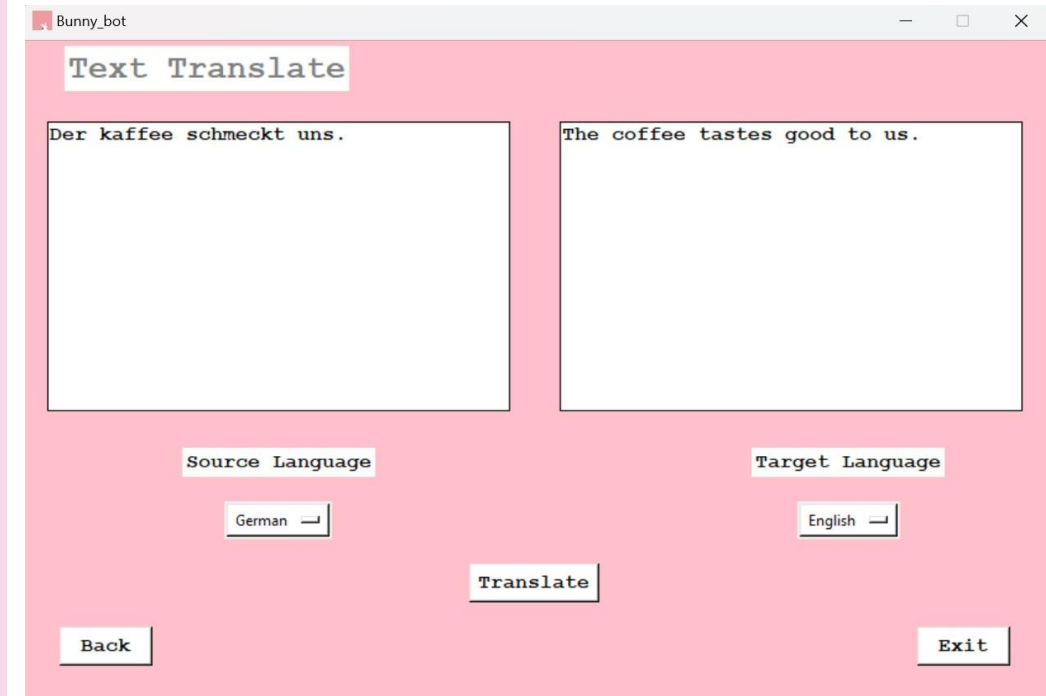
9-Create a label for selecting the target language.

10-Create a variable (target_language_var) to hold the selected target language.

11-Create an OptionMenu widget (target_language_menu) that allows the user to select the target language from a list of options.

12-Create a "Translate" button (translate_button) that triggers the translate_text() function.

13-Create "Back" and "Exit" buttons for navigation and closing the window.



Win5

Creating the fifth window (win5) for Bunny_bot application, which serves as a notepad functionality. Here's a breakdown of what's happening:

1-Create a new tkinter window (win5) with a title, fixed size, and a pink background. This window is initially hidden with win5.withdraw().

2-Create a label (win5_label) with the title "Notepad."

3-Define a class NoteTakingApp for handling the notepad functionality. It is initialized with win5 as its parent window.

4-Within the NoteTakingApp class:

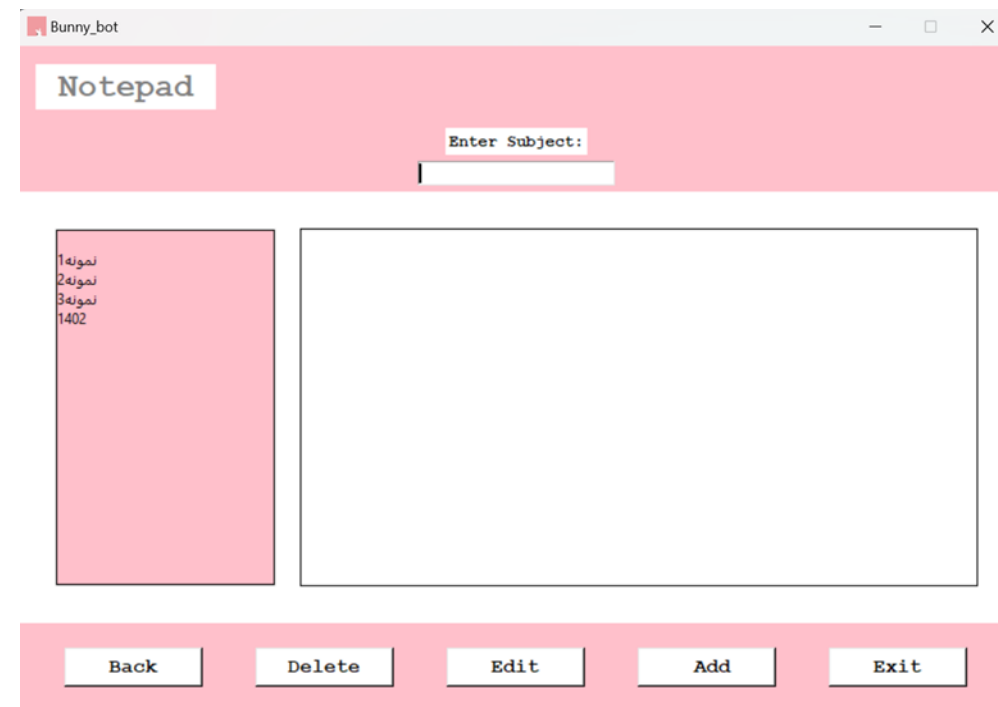
Create an entry widget for entering the subject of a note.

Create a frame for organizing the list of notes and the note text.

Create a listbox widget for displaying the list of notes.

Create a text widget for editing and viewing the note text.

Create a frame for organizing various buttons, including "Back," "Delete," "Edit," "Add," and "Exit."



Win5

5-The NoteTakingApp class has methods for:

Adding a new note.

Deleting a selected note.

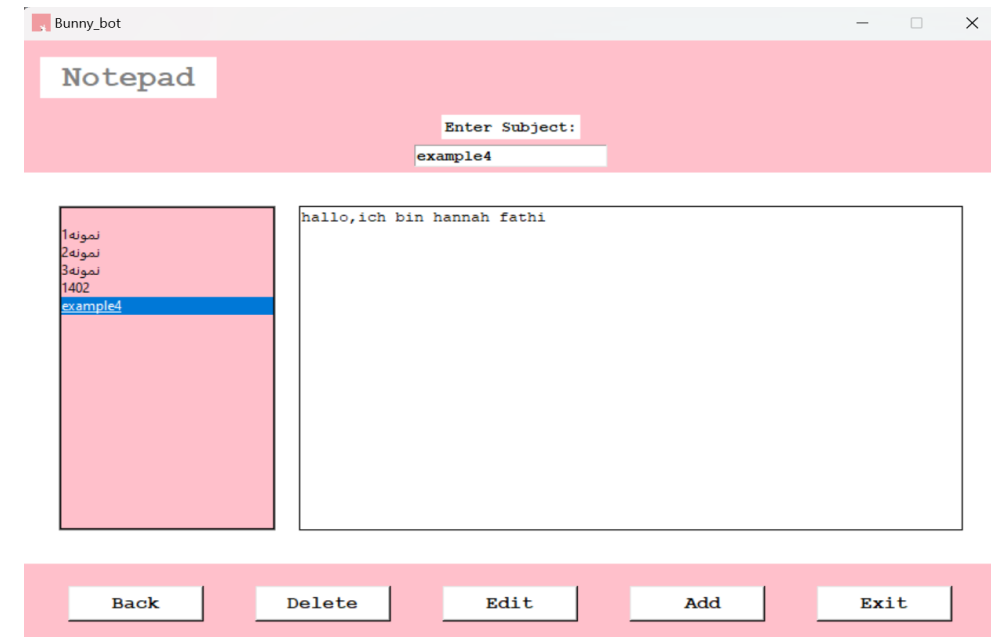
Updating the content of a selected note.

Loading saved notes from a file.

Saving notes to a file.

Handling the event when a note is selected in the list.

6-In the if `__name__ == "__main__":` block, you create an instance of the NoteTakingApp class, effectively running the notepad application.



Conclusion

The idea of Bunny_bot started to take shape in my mind from the third semester of bachelor's degree, and the first step to build and develop this program took place in the summer of 1402 with the start of the specialized project.

Bunny_bot is a conceptual concept of a multi-purpose artificial intelligence assistant that acts like a personal advisor and improves the quality of life. Bunny_bot is written in python language and uses the Openai API key to benefit from Openai services as an intelligent program engine. Bunny_bot will replace several applications and has different parts such as intelligent voice assistant using the "gpt-3.5-turbo" engine, web browser and making pictures from the text using the "image-alpha-001" engine. The translator uses the "text-davinci-003" engine and Note with the ability to save, edit and delete. The graphic design of the program has been done using the tkinter library.

Of course, Bunny_bot has a long way to reach my desired point, but the creation of this program is a milestone in the beginning of my new path as an artificial intelligence programmer.

